

UNIVERSITÉ — Polytechnique de Sousse

DOCUMENTATION TECHNIQUE

Fast Delivery

Application de gestion de commandes et livraisons

Architecture microservices

Réalise par : Khouloud Noura

1. Introduction

1.1 Contexte du projet

Ce document constitue la documentation technique complète de l'application **Fast Delivery**. Ce projet s'inscrit dans le cadre de la conception et du déploiement d'une infrastructure moderne basée sur le paradigme des architectures orientées services (SoA).

L'objectif est de concevoir et développer une application basée sur une architecture microservices en utilisant uniquement **Node.js**, respectant les contraintes suivantes : gRPC, REST, GraphQL, Kafka, bases de données séparées, et conteneurisation Docker.

1.2 Description de l'application

Fast Delivery est une application de gestion de commandes et de livraisons en temps réel. Elle permet à un client de :

- Créer une commande via une interface REST ou GraphQL
- Suivre l'état de sa livraison en temps réel
- Recevoir des notifications automatiques déclenchées par les événements métier
- Consulter un dashboard agrégé regroupant commande, livraison et notifications

1.3 Technologies utilisées

Composant	Technologie
Langage backend	Node.js (CommonJS)
Frontend	Angular 17+
Communication sync.	gRPC + Protobuf
Comm. async.	Apache Kafka (KafkaJS)
API Gateway	Express.js
API REST	Express.js
API GraphQL	graphql-http
Base SQL	SQLite3 (better-sqlite3)
Base NoSQL	RxDB (storage-memory)
Broker Kafka	Confluent Platform (Docker)
Conteneurisation	Docker + Docker Compose

2. Architecture du système

2.1 Vue globale

L'application est organisée autour d'une **API Gateway** centrale qui expose les interfaces REST et GraphQL vers le client Angular. Elle délègue les traitements aux microservices via gRPC. Les microservices communiquent entre eux de manière asynchrone via **Apache Kafka**.

2.2 Organisation des flux

L'architecture se découpe en plusieurs blocs fonctionnels connectés :

- **Frontend Angular (Port 4200)** : Communique avec la Gateway via HTTP REST/GraphQL.
- **API Gateway (Port 3000)** : Point d'entrée qui distribue vers les services gRPC.

- **Order Service (Port 50051)** : Gère la base SQLite des commandes.
- **Delivery Service (Port 50052)** : Gère la base SQLite des livraisons.
- **Notif Service (Port 50053)** : Service réactif NoSQL (RxDB).
- **Bus Kafka (Port 9092)** : Assure l'échange d'événements asynchrones.

2.3 Composants et ports

Composant	Rôle	Port
Frontend Angular	Interface utilisateur	4200
API Gateway	Point d'entrée REST + GraphQL	3000
Order Service	Gestion des commandes — gRPC	50051
Delivery Service	Gestion des livraisons — gRPC	50052
Notif Service	Gestion des notifications — gRPC	50053
Apache Kafka	Bus d'événements asynchrones	9092
Zookeeper	Coordinateur Kafka	2181

2.4 Flux de communication

Le flux général d'une requête dans le système :

1. Le client Angular envoie une requête HTTP (REST ou GraphQL) vers l'API Gateway
2. L'API Gateway traduit la requête et appelle le microservice via **gRPC**
3. Le microservice traite la logique métier, accède à sa base de données, retourne la réponse
4. Si l'action génère un événement métier, le microservice publie un message **Kafka**
5. Les microservices consommateurs traitent l'événement de manière asynchrone et indépendante

3. Description des microservices

3.1 Order Service

Le service de gestion des commandes est le point de départ du flux métier. Il est responsable du cycle de vie complet des commandes.

3.1.1 Responsabilités

- Créer une commande avec statut initial **EN_ATTENTE**
- Consulter, modifier et supprimer une commande
- Publier l'événement **commande-creee** sur Kafka après chaque création

3.1.2 Interface gRPC exposée

Listing 1 – order.proto — Service

```
service OrderService {
  rpc CreateOrder (CreateOrderRequest) returns (OrderResponse);
  rpc GetOrder    (GetOrderRequest)    returns (OrderResponse);
  rpc UpdateOrder (UpdateOrderRequest) returns (OrderResponse);
  rpc DeleteOrder (DeleteOrderRequest) returns
    (DeleteOrderResponse);
}
```

3.1.3 Base de données — SQLite3 (orders.db)

Colonne	Type	Description
orderId	TEXT (PK)	Identifiant unique généré (ORD-timestamp)
clientId	TEXT	Identifiant du client
adresse	TEXT	Adresse de livraison
items	TEXT	Liste des articles commandés
status	TEXT	EN_ATTENTE / EXPEDIEE / LIVREE

3.2 Delivery Service

Le service de livraison gère l'assignation des livreurs et le suivi des livraisons. Il est entièrement découplé de l'Order Service : il crée automatiquement une entrée de livraison en consommant l'événement Kafka `commande-creee`.

3.2.1 Responsabilités

- Écouter le topic Kafka `commande-creee` et créer une livraison automatiquement
- Assigner un livreur à une commande existante
- Mettre à jour le statut d'une livraison
- Publier les événements `livreur-assigne` et `commande-livree` sur Kafka

3.2.2 Interface gRPC exposée

Listing 2 – delivery.proto — Service

```
service DeliveryService {
  rpc AssignDelivery (AssignDeliveryRequest) returns
    (DeliveryResponse);
  rpc GetDelivery    (GetDeliveryRequest)    returns
    (DeliveryResponse);
  rpc UpdateStatus   (UpdateStatusRequest)   returns
    (DeliveryResponse);
}
```

3.2.3 Base de données — SQLite3 (deliveries.db)

Colonne	Type	Description
deliveryId	TEXT (PK)	Identifiant unique (DEL-timestamp)
orderId	TEXT	Référence à la commande associée
clientId	TEXT	Identifiant du client
livreurId	TEXT	Identifiant du livreur assigné
adresse	TEXT	Adresse de livraison
status	TEXT	A_ASSIGNER / ASSIGNE / LIVRE

3.3 Notif Service

Le service de notifications est entièrement piloté par les événements Kafka. Il ne reçoit aucun appel direct pour créer des notifications — celles-ci sont générées automatiquement lors des événements métier.

3.3.1 Responsabilités

- Écouter les topics Kafka `livreur-assigne` et `commande-livree`
- Créer automatiquement des notifications pour les clients concernés

— Exposer une interface gRPC pour la consultation des notifications

3.3.2 Interface gRPC exposée

Listing 3 – notif.proto — Service

```
service NotifService {
  rpc SendNotif (SendNotifRequest) returns (NotifResponse);
  rpc GetNotifs (GetNotifsRequest) returns (NotifsResponse);
}
message NotifsResponse { repeated NotifResponse notifs = 1; }
```

3.3.3 Base de données — RxDB (NoSQL, in-memory)

Champ	Type	Description
notifId	string (PK)	Identifiant unique (NOT-timestamp-random)
clientId	string	Identifiant du client destinataire
message	string	Contenu textuel de la notification
type	string	ASSIGNATION / LIVRAISON

Le Notif Service utilise RxDB avec stockage mémoire (`getRxStorageMemory`), justifiant l'utilisation d'une technologie NoSQL réactive. Les notifications sont des données temporelles qui n'ont pas besoin de persistance longue durée — le modèle événementiel de RxDB est parfaitement adapté.

4. API Gateway

L'API Gateway est le **point d'entrée unique** de l'application. Elle expose deux interfaces au client : REST et GraphQL. Elle ne contient aucune logique métier — elle traduit les requêtes entrantes en appels gRPC vers les microservices appropriés.

4.1 Endpoints REST

4.1.1 Commandes

Méthode	Endpoint	Description	Body
POST	/orders	Créer une commande	clientId, adresse, items
GET	/orders/ :orderId	Consulter	—
PUT	/orders/ :orderId	Modifier le statut	status
DELETE	/orders/ :orderId	Supprimer	—

4.1.2 Livraisons

Méthode	Endpoint	Description	Body
POST	/deliveries/assign	Assigner un livreur	orderId, livreurId
GET	/deliveries/ :deliveryId	Consulter	—
PATCH	/deliveries/ :id/status	Modifier le statut	status

4.1.3 Notifications

Méthode	Endpoint	Description
GET	/notifs/ :clientId	Récupérer les notifications d'un client

4.1.4 Exemple de requête REST

Listing 4 – Créer une commande — curl

```
curl -X POST http://localhost:3000/orders \
-H "Content-Type: application/json" \
-d '{"clientId": "client_01",
  "adresse": "Tunis, Centre Ville",
  "items": "Pizza, Fanta"}'
```

Listing 5 – Réponse JSON

```
{
  "orderId": "ORD-17000000000000",
  "clientId": "client_01",
  "adresse": "Tunis, Centre Ville",
  "items": "Pizza, Fanta",
  "status": "EN_ATTENTE"
}
```

4.2 API GraphQL

Endpoint : POST <http://localhost:3000/graphql>

4.2.1 Schéma GraphQL

Listing 6 – schema.gql

```
type Order { orderId: String; clientId: String
             adresse: String; items: String; status: String }
type Delivery { deliveryId: String; orderId: String
                livreurId: String; adresse: String; status:
                String }
type Notif { notifId: String; clientId: String
             message: String; type: String }
type ClientDashboard {
  order: Order; delivery: Delivery; notifs: [Notif] }

type Query {
  getOrder(orderId: String!): Order
  getDelivery(deliveryId: String!): Delivery
  getNotifs(clientId: String!): [Notif]
  dashboard(orderId: String!, deliveryId: String!,
            clientId: String!): ClientDashboard
}

type Mutation {
  createOrder(clientId: String!, adresse: String!,
              items: String!): Order
  assignDelivery(orderId: String!, livreurId: String!,
                 adresse: String!): Delivery
  updateOrderStatus(orderId: String!, status: String!): Order
  updateDeliveryStatus(deliveryId: String!, status: String!):
    Delivery
}
```

4.2.2 Requête Dashboard — exemple

Listing 7 – Query GraphQL — Dashboard agrégé

```
query GetDashboard($orderId: String!, $deliveryId: String!,
                  $clientId: String!) {
  dashboard(orderId: $orderId, deliveryId: $deliveryId,
            clientId: $clientId) {
    order      { orderId status items adresse }
    delivery   { deliveryId status livreurId adresse }
    notifs     { message type }
  }
}
```

4.2.3 Justification de GraphQL

GraphQL est utilisé principalement pour le **dashboard client** qui nécessite des données de 3 microservices différents. Avec REST, cela nécessiterait 3 requêtes HTTP séquentielles. Avec GraphQL :

- Une seule requête côté client
- L'API Gateway effectue les 3 appels gRPC en parallèle (`Promise.all`)
- Le client demande exactement les champs nécessaires, sans sur-fetch

5. Communication gRPC

La communication gRPC est l'élément technique central du projet. Tous les échanges entre l'API Gateway et les microservices passent exclusivement par gRPC sur **HTTP/2**, garantissant des performances élevées et une sérialisation efficace via **Protobuf**.

5.1 Fichier order.proto

Listing 8 – proto/order.proto

```
syntax = "proto3";
package order;

service OrderService {
  rpc CreateOrder (CreateOrderRequest) returns (OrderResponse);
  rpc GetOrder    (GetOrderRequest)    returns (OrderResponse);
  rpc UpdateOrder (UpdateOrderRequest) returns (OrderResponse);
  rpc DeleteOrder (DeleteOrderRequest) returns
    (DeleteOrderResponse);
}

message CreateOrderRequest {
  string clientId = 1;
  string adresse  = 2;
  string items    = 3;
}

message OrderResponse {
  string orderId = 1; string clientId = 2;
  string adresse = 3; string items    = 4; string status = 5;
}

message DeleteOrderResponse { string message = 1; }
```

5.2 Fichier delivery.proto

Listing 9 – proto/delivery.proto

```
syntax = "proto3";
package delivery;

service DeliveryService {
  rpc AssignDelivery (AssignDeliveryRequest) returns
    (DeliveryResponse);
  rpc GetDelivery    (GetDeliveryRequest)    returns
    (DeliveryResponse);
  rpc UpdateStatus   (UpdateStatusRequest)   returns
    (DeliveryResponse);
}

message AssignDeliveryRequest {
  string orderId      = 1;
  string livreurId    = 2;
  string adresse      = 3;
}

message DeliveryResponse {
  string deliveryId = 1; string orderId = 2;
  string livreurId  = 3; string adresse = 4; string status = 5;
}
```

5.3 Fichier notif.proto

Listing 10 – proto/notif.proto

```
syntax = "proto3";
package notif;

service NotifService {
  rpc SendNotif (SendNotifRequest) returns (NotifResponse);
  rpc GetNotifs (GetNotifsRequest) returns (NotifsResponse);
}

message SendNotifRequest {
  string clientId = 1; string message = 2; string type = 3;
}

message NotifsResponse { repeated NotifResponse notifs = 1; }
```

5.4 Appel gRPC depuis l'API Gateway

L'API Gateway encapsule chaque appel gRPC dans une Promise pour une intégration propre avec Express.js et GraphQL :

Listing 11 – api-gateway/index.js — wrapper gRPC

```
function grpcCall(client, method, data) {
  return new Promise((resolve, reject) => {
    client[method](data, (err, response) => {
      if (err) reject(err);
      else resolve(response);
    });
  });
}
```



```

    });
  });
}

// Exemple d'utilisation dans un endpoint REST
app.post('/orders', async (req, res) => {
  const response = await grpcCall(
    orderClient, 'createOrder', req.body
  );
  res.status(201).json(response);
});

```

5.5 Gestion des erreurs gRPC

Chaque microservice retourne des codes d'erreur gRPC standardisés :

Code gRPC	Situation
NOT_FOUND	Ressource inexistante (commande, livraison)
INTERNAL	Erreur base de données ou traitement interne
ALREADY_EXISTS	Livraison déjà créée pour cette commande

6. Communication Kafka

Kafka est utilisé pour la communication événementielle entre les microservices. Il permet de les **découpler complètement** : aucun microservice n'appelle directement un autre.

6.1 Topics Kafka

Topic	Producteur	Consommateur	Déclencheur
commande-creee	Order Service	Delivery Service	Création commande
livreur-assigne	Delivery Service	Notif Service	Assignment livreur
commande-livree	Delivery Service	Notif Service	Statut = LIVRE

6.2 Scénarios métier

6.2.1 Scénario 1 — Création d'une commande

1. Le client soumet une commande via Angular (POST /orders)
2. L'API Gateway appelle `createOrder` sur l'Order Service via gRPC
3. L'Order Service insère en SQLite et publie sur `commande-creee`
4. Le Delivery Service consomme et crée une livraison avec statut `A_ASSIGNER`

6.2.2 Scénario 2 — Assignment d'un livreur

1. L'administrateur assigne un livreur via l'interface Angular
2. Le Delivery Service met à jour la livraison et publie sur `livreur-assigne`
3. Le Notif Service crée la notification : « *Votre commande a été prise en charge par un livreur* »

6.2.3 Scénario 3 — Livraison effectuée

1. Le statut est mis à jour à `LIVRE`
2. Le Delivery Service publie sur `commande-livree`

3. Le Notif Service crée la notification : « *Votre commande a été livrée !* »

6.3 Format des messages Kafka

Listing 12 – Topic : commande-creee

```
{
  "orderId": "ORD-17000000000000",
  "clientId": "client_01",
  "adresse": "Tunis, Centre Ville",
  "items": "Pizza, Fanta",
  "status": "EN_ATTENTE"
}
```

Listing 13 – Topic : livreur-assigne

```
{
  "deliveryId": "DEL-17000000000001",
  "orderId": "ORD-17000000000000",
  "clientId": "client_01",
  "livreurId": "livreur_01",
  "status": "ASSIGNE"
}
```

7. Frontend Angular

Le frontend est développé avec Angular 17+. Il communique exclusivement avec l'**API Gateway** via HTTP — aucun accès direct aux microservices gRPC ou à Kafka.

7.1 Composants

Composant	Protocole	Description
CreateOrder	REST POST /orders	Formulaire création commande
OrderManagement	REST GET/PUT/DELETE	Recherche, modif., suppression
AssignDelivery	REST POST /deliveries/assign	Assignment livreur
Notifications	REST GET /notifs (poll. 3s)	Flux notifications temps réel
Dashboard	GraphQL query dashboard	Vue agrégée des 3 services

7.2 Service API centralisé

Toutes les communications HTTP sont centralisées dans **ApiService**. Ce service encapsule les appels REST et GraphQL, isolant les composants de la logique réseau.

Listing 14 – src/app/services/api.ts — extrait

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = 'http://localhost:3000';

  createOrder(order: any): Observable<any> {
    return this.http.post(`${this.baseUrl}/orders`, order);
  }
}
```

```

getDashboard(orderId: string, deliveryId: string, clientId:
  string): Observable<any> {
  const query = {
    query: 'query GetDashboard(...) { ... }',
    variables: { orderId, deliveryId, clientId }
  };
  return this.http.post(`${this.baseUrl}/graphql`, query)
    .pipe(map((res: any) => res.data.dashboard));
}
}

```

7.3 Notifications en temps réel

Le composant Notifications implémente un **polling toutes les 3 secondes** via RxJS (`interval + switchMap`). L'utilisateur entre son `clientId` et le flux se déclenche automatiquement :

Listing 15 – Polling RxJS — Notifications

```

this.pollingSub = interval(3000).pipe(
  startWith(0),
  switchMap(() => this.api.getNotifications(this.clientId))
).subscribe({ next: (res) => this.notifs = res.notifs || [] });

```

8. Conteneurisation Docker

L'application est entièrement conteneurisée avec Docker et Docker Compose. Chaque microservice dispose de son propre Dockerfile. Le fichier `docker-compose.yml` orchestre l'ensemble des 7 services.

8.1 Services Docker Compose

Service	Image / Build	Dépendances
zookeeper	cp-zookeeper :7.5.0	—
kafka	cp-kafka :7.5.0	zookeeper
order-service	Build local	kafka (healthy)
delivery-service	Build local	kafka (healthy)
notif-service	Build local	kafka (healthy)
api-gateway	Build local	order, delivery, notif

8.2 Variables d'environnement

Variable	Valeur	Utilisée par
KAFKA_BROKER	kafka :9092	Tous les services
ORDER_SERVICE_URL	order-service :50051	API Gateway
DELIVERY_SERVICE_URL	delivery-service :50052	API Gateway